

UniTrack

Academic Advising System

User Guide — Version 2.0

Revision Sheet

Version	Date	Author	Description of Change				
1.0	Fall 2024	Dev Team	Initial release — core GPA and record management	2.0	Spring 2026	Dev Team	Added degree audit, alerts, schedule calendar, and registration

Table of Contents

Revision Sheet	2
Welcome	5
Introduction	5
System Overview	5
Project References	5
Where to Get Help	6
Organization of This Manual	6
Acronyms and Abbreviations	6
What's New in Version 2.0	8
System Summary	9
System Requirements and Platform Support	9
Building and Installing	9
Data and Function Flow	10
Caveats and Exceptions	10
Using the System	11
Starting the App	11
Home Page — Degree Progress	11
Schedule Page	11
Registration Page	12
Alerts Page	12
Exiting the App	13
Administration — Maintaining the System	14
Saving Student Data	14
Updating the Course Catalog	14
Resetting a Student Record	14
Updating the Degree Plan	14
Appendices	15
Appendix A — Class Architecture	15
Appendix B — REST API Endpoint Reference	16
Appendix C — CS Degree Requirements Summary	17
References	17

Welcome

Welcome to UniTrack — the academic advising system built specifically for Computer Science students at Midwestern State University. UniTrack brings together your course history, degree requirements, class schedule, and registration all in one place, so you always know exactly where you stand on the path to graduation.

This guide covers everything you need to get started, navigate each section of the application, and understand the alerts and recommendations the system generates on your behalf.

Introduction

System Overview

UniTrack is a locally-hosted web application that runs a C++ backend server alongside a set of HTML/JavaScript frontend pages. The backend loads the MSU course catalog from a CSV file, maintains your academic record in memory, and exposes a REST API on localhost:8080. The frontend connects to that API to display and update your information in real time.

The system covers four primary functions:

- Degree Audit — tracks completion of all CS degree requirements including rigid core courses, category requirements, and flexible general education credits.
- Schedule — shows your enrolled courses in a weekly calendar view and a sortable list view, with optional GPA projection.
- Registration — lets you browse the full course catalog, see which courses you are eligible to take next, and add them to your record.
- Alerts — surfaces actionable notifications such as unmet prerequisites, GPA warnings, missing required courses, and newly available courses.

Project References

The following documents and resources are related to this project:

- MSU Texas Computer Science Degree Plan — official course requirements.
- Schedule_with_Credits.csv — course catalog with CRN, subject, code, credits, title, days, and meeting times.
- Degree_Plan.csv — structured degree requirements used to seed the application.
- student_data.txt — local persistence file where your academic record is saved.
- cpp-httpplib — single-header C++ HTTP server library (httpplib.h).

Where to Get Help

If you encounter an issue with UniTrack, try the following steps in order:

1. Confirm the C++ server is running and listening on port 8080. Open a terminal and check that the server process is active.
2. Open your browser's developer console (F12) and look for any network errors on the localhost:8080 requests.
3. Verify that Schedule_with_Credits.csv is present in the same directory as the compiled server executable.
4. Check student_data.txt for any corrupted or malformed lines if your record fails to load.
5. Contact the development team or your course instructor if the problem persists.

Organization of This Manual

This manual is organized into the following sections:

- System Summary — requirements, installation, and data flow.
- Using the System — detailed walkthrough of each page with expected inputs and outputs.
- Administration — saving data and maintaining the system.
- Appendices — architecture reference, API endpoint table, and alert type reference.
- References — tools and libraries used in the project.

Acronyms and Abbreviations

Term	Definition
API	Application Programming Interface — the set of HTTP endpoints the frontend calls to read and write data.
CRN	Course Registration Number — a unique identifier for a specific section of a course.
CSV	Comma-Separated Values — plain-text file format used for the course catalog.
GPA	Grade Point Average — weighted average of grade points earned per credit hour.

JSON	JavaScript Object Notation — text format used to exchange data between the C++ backend and HTML frontend.
MSU	Midwestern State University — the institution this application was built for.
REST	Representational State Transfer — architectural style for HTTP APIs used by the backend.

What's New in Version 2.0

Version 2.0 is a major update over the initial release. The following features have been added or significantly improved:

- Degree Audit (Home page) — Full audit of CS degree requirements including rigid core courses, Math upper elective, and all six general education flex categories. Shows per-category progress bars and expandable course lists.
- Alerts page — Six distinct alert types with color-coded severity cards (critical, warning, informational), dismiss/restore support, and per-alert action buttons that navigate directly to the relevant page.
- Schedule page — Weekly calendar view and list view showing enrolled courses with meeting days, times, and room numbers. Includes a live projected GPA display that updates when what-if grades are entered.
- Registration page — Browse and search the full course catalog. Recommended courses (those you are eligible for next) appear at the top. Add a course to your record with a single click; the system checks for time conflicts before confirming.
- Flex category assignment — On the Home page, students can assign an approved elective to each general education flex category. The system enforces the no-reuse rule and surfaces a COURSE_REUSED alert if a conflict is detected.
- Section scheduling data — Meeting days, start/end times, room number, and professor are now parsed from the CSV and displayed throughout the application.
- Persistent save — Student data is written to student_data.txt on demand via the Save button and reloaded automatically on server startup.

System Summary

System Requirements and Platform Support

UniTrack is a desktop web application. The following components are required to run it:

Component	Requirement
Operating System	Windows 10/11 (compiled with MSVC). The application uses Windows SDK headers (windows.h, SDKDDKVer.h).
C++ Compiler	Microsoft Visual C++ (MSVC) — the project targets the Windows platform via framework.h and targetver.h.
Web Browser	Any modern browser (Chrome, Firefox, Edge) that supports ES6 JavaScript and the Fetch API.
Network	No internet connection required. All communication is on localhost:8080.
Disk Space	Minimal — the compiled executable plus the CSV catalog file and student_data.txt.
RAM	Minimal — the full course catalog and student record are held in memory but are small in size.

Building and Installing

6. Open the project solution in Visual Studio.
7. Ensure the following files are present in the project directory: all .h and .cpp source files, httpLib.h, Schedule_with_Credits.csv, and Degree_Plan.csv.
8. Build the solution in Release or Debug configuration.
9. Run the compiled executable from the project directory (so relative file paths to the CSV files resolve correctly).
10. Open a web browser and navigate to home.html — the frontend will connect to localhost:8080 automatically.

Note

The server must be running before opening any HTML page. If the browser shows 'Network error — is the server running?', verify the executable is active and listening on port 8080.

Data and Function Flow

The diagram below summarizes how data moves through UniTrack:

Data Flow

1. Server startup → `Course::loadFromFile()` reads `Schedule_with_Credits.csv` into memory
2. Server startup → `loadStudentFromFile()` reads `student_data.txt` and rebuilds the Student object
3. Browser request → Frontend calls API endpoint (e.g. `GET /student`)
4. Server handler → Reads/writes the in-memory Student or Course objects
5. Server response → Serializes result to JSON via `toJson()` and returns HTTP 200
6. Frontend render → JavaScript reads JSON and updates the page DOM
7. Save → `POST /save` triggers `saveStudentToFile()` writing `student_data.txt`

Caveats and Exceptions

- All data is in memory. If the server crashes without a save, unsaved changes are lost.
- The degree plan is hardcoded for the MSU Texas Computer Science program. Changing majors requires a code update.
- Grade comparison for prerequisites uses numeric GPA equivalents. An unrecognized grade string returns 0.0 and will fail any minimum-grade check.
- The course catalog is read once at startup. Adding new courses requires a server restart.
- Time conflict detection requires that both sections have parseable meeting times in the CSV. Sections with empty days or times are treated as non-conflicting.

Using the System

Starting the App

11. Compile and run the C++ server executable from the project directory.
12. Confirm the terminal shows a startup message indicating the server is listening on port 8080.
13. Open home.html in your web browser.
14. If this is your first time launching the application and no student_data.txt exists, a setup modal will appear prompting you to enter your name and major.

Home Page — Degree Progress

The Home page is your dashboard. At the top you will see your name, major, and current GPA. Below that is a two-color progress bar showing credits completed (green) and credits in progress (blue) out of the 120 required to graduate.

Below the progress bar, each degree requirement category is listed as a collapsible row. Clicking a row expands it to show the individual courses in that category along with their completion status. Categories include:

- Core CMPS Courses — the mandatory computer science sequence.
- Mathematics — Pre-Calculus, Calculus I, and the Math upper elective.
- Physics — Physics I and Physics II.
- General Education (fixed) — English Composition, History, and Political Science.
- General Education (flex) — Communications, Lab Science, Language/Philosophy/Culture, Creative Arts, Social/Behavioral Sciences, and Cultural/Global Understanding. For flex categories, a dropdown lets you assign an approved elective course.

Expected input: Clicking a flex category dropdown and selecting a course from the approved list.

Expected output: The selected course is sent to POST /flex. If accepted, the category shows as satisfied. If the course was already used for another category, a COURSE_REUSED alert is generated.

Schedule Page

The Schedule page shows all courses currently in your academic record. Two views are available via tabs at the top:

- **Weekly Calendar** — a Mon–Fri grid showing each course as a colored block at its meeting time. Hovering a block reveals the course name, room, and professor.
- **Course List** — a card-based list of all enrolled courses with grade, semester, credit hours, meeting days/times, and a Drop button.

A projected GPA value is displayed in the top right. In-progress courses without a grade do not count toward the GPA. You can enter a what-if grade for any in-progress course in the list view to see how it would affect your GPA.

Expected input: Clicking 'Drop' on a course card, or typing a grade into the grade field of an in-progress course.

Expected output: Drop sends DELETE /record. The course disappears from the schedule and the GPA updates. Entering a grade sends PUT /record and the projected GPA updates immediately.

Registration Page

The Registration page lets you search the MSU course catalog and add courses to your record. The page opens with two sections:

- **Recommended Courses** — courses you are eligible to take next (all prerequisites satisfied, not yet completed). These are highlighted at the top with a 'Recommended' badge.
- **All Courses** — the full catalog organized by department. Click a department header to expand its courses and their available sections.

Each section row shows the CRN, meeting days, time, professor, and room. Clicking 'Add' opens a confirmation modal asking for the semester and whether the course is already completed.

Expected input: Selecting a course section and clicking 'Add', then filling in the semester field and completion status.

Expected output: The system sends POST /record. If a time conflict is detected with an existing enrolled course, the server returns an error message and the course is not added. On success, the course appears in your schedule and the degree audit updates.

Alerts Page

The Alerts page lists all active system notifications for your account. Alerts are generated automatically each time the degree audit runs (on every page load). Each alert card shows:

- A colored severity indicator (red = critical, orange = warning, blue = informational).

- A short title describing the issue.
- A detailed message explaining exactly what triggered the alert.
- An action button that navigates directly to the relevant page (e.g., 'View Degree Audit' or 'Register Now').
- A 'Dismiss' button to hide the alert. Dismissed alerts appear in a collapsed section at the bottom of the page.

Alert Type	Severity	Meaning
GPA_WARNING	Critical	Your cumulative GPA has dropped below the 2.0 minimum required for the CS program.
PREREQ_NOT_MET	Critical	You are enrolled in or attempting a course whose prerequisite you have not completed with the required minimum grade.
MANDATORY_NOT_MET	Warning	A course that is specifically required by your degree plan is missing from your record with no substitution allowed.
HOURS_NOT_MET	Warning	You have not yet accumulated enough credit hours in a required category.
ELIGIBLE_COURSE	Informational	All prerequisites for this course are now satisfied and it is available for you to register.
COURSE_REUSED	Warning	You attempted to assign the same course to satisfy two different degree requirements. Each course may only satisfy one requirement.

Expected input: Clicking 'Dismiss' on an alert card.

Expected output: The alert card fades out and moves to the Dismissed section. Clicking 'Restore All' on the dismissed section header brings all dismissed alerts back.

Exiting the App

15. Click the Save button (if visible) or trigger POST /save from the browser to persist your current record to student_data.txt.
16. Close the browser tab.
17. Stop the C++ server process in the terminal (Ctrl+C or close the terminal window).

Important

Always save before stopping the server. The in-memory student record is not automatically persisted — unsaved changes will be lost when the server process exits.

Administration — Maintaining the System

Saving Student Data

Student data is saved to `student_data.txt` using a pipe-delimited format. Each course record occupies one line:

```
CMPS1044 | Computer Science I | A | 4 | Fall 2024 | Completed
```

To trigger a save, call `POST /save` through the browser (a Save button on the page does this automatically) or via any HTTP client. The file is reloaded automatically the next time the server starts.

Updating the Course Catalog

18. Open `Schedule_with_Credits.csv` in a spreadsheet application or text editor.
19. Add or modify rows following the column format: CRN, Subject, Code, Credits, Title, Days, StartTime, EndTime, RoomNumber (optional), Professor (optional).
20. Save the file and restart the server. The catalog is reloaded from disk on every startup.

Resetting a Student Record

21. Stop the server.
22. Delete or clear the contents of `student_data.txt`.
23. Restart the server. The setup modal will appear again on the next page load.

Updating the Degree Plan

The CS degree requirements are hardcoded in `DegreePlan.cpp` inside the `loadCSDegreePlan()` method. To modify requirements:

- Rigid courses — add or remove entries from the `rigidCourses` vector.
- Prerequisite rules — add or modify entries in the `prereqRules` map using the `PrereqRule` struct.
- Flex categories — add or modify `FlexCategory` entries in the `flexCategories` vector, updating the approved course list as needed.

After any change to `DegreePlan.cpp`, rebuild the project and restart the server.

Appendices

Appendix A — Class Architecture

The following table summarizes each class or major file and its responsibility within the system:

Class / File	Responsibility
Student	Core entity. Owns the list of AcademicRecords and Alerts. Computes GPA. Serializes to JSON.
AcademicRecord	One row in a student's course history. Ties a Course to a semester, grade, and completion status. Provides grade-to-GPA conversion.
Course	One course offering from the catalog CSV. Owns a Section. Parses CSV lines via fromCSV().
Section	Meeting details for a course: professor, days, start/end times, room. Parsed from CSV columns 5-8.
DegreePlan	Hardcoded CS degree rulebook: rigid courses, category requirements, flex categories, and prereq staircase. Same for every student.
DegreeAudit	Referee. Reads DegreePlan rules and checks a student's records against every requirement. Produces fulfilled/unfulfilled lists, category progress, alerts, and eligible courses.
Alert / AlertType	One system notification. Generated by DegreeAudit. Six types: GPA_WARNING, PREREQ_NOT_MET, MANDATORY_NOT_MET, HOURS_NOT_MET, ELIGIBLE_COURSE, COURSE_REUSED.
Structs.h	Shared plain-data structs: PrereqRule, CategoryRequirement, FlexCategory, CategoryProgress, FlexProgress.
Server.cpp	Application entry point. Starts the http lib server on port 8080 and defines all HTTP routes.
api.js	Frontend API connector. Wraps fetch() calls to every backend endpoint. Returns parsed JSON or an error object.
nav.js	Shared sidebar navigation controller. Highlights the active page and wires click handlers.
home.html	Degree progress dashboard. Shows GPA, credit progress bar, and expandable requirement category rows.
schedule.html	Weekly calendar and list view of enrolled courses. Supports drop and what-if grade entry.
registration.html	Course catalog browser. Shows recommended courses at top. Adds courses with conflict detection.

alerts.html	Notification center. Displays color-coded alert cards with action buttons and dismiss support.
-------------	--

Appendix B — REST API Endpoint Reference

All endpoints are served on `http://localhost:8080`. Request and response bodies are JSON.

Method	Endpoint	Description
GET	<code>/status</code>	Health check. Returns <code>{ status: 'ok' }</code> .
GET	<code>/student</code>	Returns the full student object including records, GPA, credit hours, and alerts.
POST	<code>/student</code>	Creates the student profile (first-time setup). Body: <code>{ name, major }</code> .
PUT	<code>/student</code>	Updates student name or major. Body: <code>{ name, major }</code> .
GET	<code>/courses</code>	Returns the full course catalog array from the CSV.
GET	<code>/audit</code>	Runs DegreeAudit and returns fulfilled, unfulfilled, category progress, flex progress, eligible courses, alerts, and completion percentage.
GET	<code>/alerts</code>	Returns the student's active alert list.
POST	<code>/record</code>	Adds a course to the academic record. Body: <code>{ courseCode, grade, semester, completed }</code> .
PUT	<code>/record</code>	Updates a grade for an existing record. Body: <code>{ courseCode, grade }</code> .
DELETE	<code>/record</code>	Removes a course from the academic record. Body: <code>{ courseCode }</code> .
POST	<code>/flex</code>	Assigns a course to a flex gen-ed category. Body: <code>{ courseCode, categoryCode }</code> .
POST	<code>/alerts/dismiss</code>	Dismisses an alert by type/courseCode/message fingerprint.
POST	<code>/alerts/restore</code>	Restores all dismissed alerts.
POST	<code>/save</code>	Persists the current student record to <code>student_data.txt</code> .

Appendix C — CS Degree Requirements Summary

The following rigid courses must be completed by every CS student with no substitutions:

- Core CMPS: CMPS1044, CMPS1063, CMPS2084, CMPS2143, CMPS2433, CMPS3013, CMPS3023, CMPS3233, CMPS4143, CMPS4103, CMPS4113, CMPS4991
- Mathematics: MATH1534, MATH1634 (plus one of MATH2753, MATH3533, or MATH4243)
- Physics: PHYS1624, PHYS2644
- General Education (fixed): ENGL1143, HIST1133, HIST1233, POLS1333, POLS1433

Six flex gen-ed categories must also be satisfied by choosing approved electives:

- Communications (010B) — 3 hours
- Lab Science (030N) — 6 hours
- Language, Philosophy, and Culture (040N) — 3 hours
- Creative Arts (050N) — 3 hours
- Social and Behavioral Sciences (080N) — 3 hours
- Cultural and Global Understanding (090A) — 3 hours

Advanced CMPS electives: 15 additional credit hours of 4000-level CMPS courses beyond the mandatory core.

References

The following tools, libraries, and resources were used in the development of UniTrack:

- `cpp-http-lib` — a single-header C++ HTTP/HTTPS server and client library. Used to implement the REST API backend in `Server.cpp`. Available at <https://github.com/yhirose/cpp-http-lib>.
- DM Sans & DM Mono — open-source typefaces by Colophon Foundry, served via Google Fonts. Used in all HTML pages for the interface typography.
- MSU Texas Computer Science Degree Plan — the official academic requirements document from the MSU Texas Registrar's Office used to hardcode the degree rules in `DegreePlan.cpp`.
- Microsoft Visual C++ (MSVC) — the compiler and Windows SDK used to build the application, referenced via `framework.h` and `targetver.h`.
- ECMAScript 2016 (ES6) Fetch API — the browser-native HTTP interface used in `api.js` to communicate with the backend.